

SHUNYA

Evolution Implementation Plan

2026-07-14

Based on Architecture Report 2026-07-14 + Master Architectural Evolution Directive

Prepared for: Nishesh Singhal

SHUNYA EVOLUTION IMPLEMENTATION PLAN

Based on: - SHUNYA Architecture Report 2026-07-14 (PDF 1 — Current State) - SHUNYA Master Architectural Evolution Directive 2026-07-14 (PDF 2 — Target Architecture)

Implementation Order

Per Master Directive First Execution Instruction:

PHASE 0 → PHASE 1 → PHASE 2 → PHASE 3 → PHASE 5 → PHASE 4 → PHASE 6 → PHASE 7 → PHASE 8 → PHASE 9 → PHASE 10 → PHASE 11 → PHASE 12 → PHASE 13 → PHASE 14 → PHASE 15 → PHASE 16 → PHASE 17

PHASE 0 — Safety, Baseline & Architectural Convergence

Objective: Preserve working state. Unify the two-codebase problem. Add characterization tests.

Evidence (Architecture Report §1): - Two working directories (/root/panchi_backend/, /root/shunya_os/) — NOT a monorepo - 8 uncommitted files in Panchi backend - 14 uncommitted files in Shunya OS frontend - Monolithic routes.py (1529 lines) - Shunya OS admin/dashboard/etc blueprints NOT registered in running service

Preservation Rules (Master Directive §NON-NEGOTIABLE): - knowledge_store.py — PRESERVE - KnowledgeFact versioning — PRESERVE - executor.py — PRESERVE - Multi-channel adapter architecture — PRESERVE - ontology.py — PRESERVE - Multi-vertical ontology architecture — PRESERVE - Theme engine — PRESERVE - shunya.css — PRESERVE - PWA manifest — PRESERVE - Service worker — PRESERVE - Lead pipeline — PRESERVE - Payment functionality — PRESERVE - Invoice functionality — PRESERVE

Actions:

- A. Commit all uncommitted work on both directories git tag: pre-evolution-baseline
- B. Add characterization tests: - /root/panchi_backend/tests/test_models.py — Lead creation, status updates, Payment, Invoice - /root/panchi_backend/tests/test_routes.py — ShunyaInterface.process_message(), KnowledgeStore.store()
- C. Map architectural convergence: - Map all imports - Map template loaders (Jinja2 ChoiceLoader) - Map blueprints (auth_bp, main, api, client_bp) - Map deployment paths (systemd, gunicorn, nginx) - Map database tables (22 tables) - Preserve production boot compatibility
- D. No destructive movement. Migration adapters where required.

Database: None required.

Rollback: git checkout pre-evolution-baseline

PHASE 1 — Unified Human Identity Domain

Evidence (Architecture Report §6): - No unified Person model - Lead.customer_name is a STRING, not a model - TeamMember for employees - ClientUser for portal users - Person model EXISTS in Shunya OS models but is DISCONNECTED - Same human represented differently across modules

Target (Master Directive §Phase 1): - Person as first-class entity - PersonIdentity for email, phone, WhatsApp, Telegram, application account - Identity resolution states: EXACT, VERIFIED, PROBABLE, AMBIGUOUS, UNRESOLVED - EmployeeProfile, CustomerProfile as role/relationship projections — not attributes on Person - Never silently merge ambiguous humans

New Files: - /root/panchi_backend/app/shunya/identity/person.py — Person, PersonIdentity models - /root/panchi_backend/app/shunya/identity/resolver.py — IdentityResolver - /root/panchi_backend/app/shunya/identity/init.py

Database Migration: - CREATE TABLE persons (id, tenant_id, display_name, primary_language, locale, timezone, identity_status, created_at, updated_at) - CREATE TABLE person_identities (id, person_id, identity_type, identity_value, normalized_value, verification_state) - Add nullable person_id FK to leads, team_members, client_users

Backfill: - Create Person record for each unique lead.customer_name via phone number resolution - Link existing TeamMember records to Person records

Adapter: - Lead.person — property that resolves via Lead.phone → PersonIdentity → Person - Existing code using lead.customer_name string continues to work

Preserved: - Lead model — unchanged (person_id is nullable FK) - TeamMember model — unchanged (person_id is nullable FK) - All existing queries and routes

PHASE 2 — Relationship Intelligence Domain

Evidence (Architecture Report §6, §9): - 0 references to "relationship" in shunya pipeline - Relationship model EXISTS in Shunya OS models but is DISCONNECTED - No unified connection between PERSON↔ORGANIZATION, PERSON↔PERSON

Target (Master Directive §Phase 2): - Relationship as first-class domain - Types: ORGANIZATION↔PERSON, PERSON↔PERSON, PERSON↔TEAM, PERSON↔CUSTOMER, PERSON↔SUPPLIER, PERSON↔BUSINESS ENTITY - A lead is an EVENT in a customer relationship — not the customer itself - A booking IS NOT the customer - An employee IS NOT a login account

New Files: - /root/panchi_backend/app/shunya/relationship/models.py — Relationship model - /root/panchi_backend/app/shunya/relationship/init.py

Database Migration: - CREATE TABLE relationships (id, tenant_id, subject_entity_type, subject_entity_id, relationship_type, object_entity_type, object_entity_id, status, start_at, end_at, metadata, created_at, updated_at)

PHASE 3 — Conversation & Interaction Domain

Evidence (Architecture Report §7): - AI Assistant is rule-based keyword routing — NOT LLM - No conversational entity context - No pronoun resolution ("it", "her", "that booking") - No conversation memory - No session continuity

Target (Master Directive §Phase 3): - First-class Conversation, ConversationParticipant, ConversationContext models - Entity continuity across conversation turns - Context resolution: "Ritu" → Person, "her Bali trip" → Plan, "it" → Booking, "Send it to her" → Executor action

New Files: - /root/panchi_backend/app/shunya/conversation/models.py — Conversation, ConversationParticipant, ConversationContext - /root/panchi_backend/app/shunya/conversation/context.py — ContextResolver - /root/panchi_backend/app/shunya/conversation/init.py

Database Migration: - CREATE TABLE conversations (id, tenant_id, conversation_type, status, primary_context, started_at, last_activity_at) - CREATE TABLE conversation_participants (id, conversation_id, person_id, role, channel) - CREATE TABLE conversation_contexts (id, conversation_id, entity_type, entity_id, label, active)

Note: Phase 3 wires conversations to Person (Phase 1) and Relationship (Phase 2).

PHASE 5 — Privacy, Sensitivity & Memory Eligibility

Evidence (Architecture Report §10): - NO sensitivity classification - NO consent tracking - NO retention policy - NO purpose limitation - Session vs long-term memory NOT DISTINGUISHED - Data deletion: NOT IMPLEMENTED - Health/political information: NOT HANDLED - Observation and LearningEntry persist with NO TTL

Target (Master Directive §Phase 5): - Policy boundaries: CAN_UNDERSTAND_NOW CAN_RETAIN_IN_SESSION CAN_RETAIN_TEMPORARILY CAN_REMEMBER_LONG_TERM CAN_EXPOSE_TO_ORGANIZATION CAN_USE_FOR_BUSINESS_REASONING CAN_USE_FOR_COMMERCIAL_ACTION CAN_SHARE_WITH_ANOTHER_EMPLOYEE CAN_DELETE_FORGET

- Sensitivity levels: PUBLIC, ORGANIZATION_CONFIDENTIAL, SENSITIVE, RESTRICTED
- ConsentRecord where required
- RetentionPolicy with TTL per sensitivity level

New Files: - /root/panchi_backend/app/shunya/governance/privacy.py — SensitivityClassifier, ConsentRecord, RetentionPolicy, MemoryEligibility

Database Migration: - Add sensitivity column to knowledge_facts, observations, learning_entries - CREATE TABLE consent_records - CREATE TABLE retention_policies - Add memory_eligibility to conversations, human_signals

Mandatory before Phase 4: Human context cannot persist sensitive signals without privacy boundaries.

PHASE 4 — Human Context Domain

Evidence (Architecture Report §6, §9): - mood: 0 references in shunya pipeline — MISSING - sentiment: 0 references — MISSING - preference: 0 references — MISSING - memory: 0 references — MISSING - behavioural: 0 references — MISSING - companion.py has rule-based mood detection (text templates only)

Target (Master Directive §Phase 4): - HumanSignal model with signal type classification - Temporal relevance: TEMPORARY vs REPEATED vs PATTERN_CANDIDATE - "I am tired today" → TEMPORARY SIGNAL, short temporal relevance - NOT → "Personality = TIRED" - PatternCandidate requires: sufficient evidence, temporal spread, confidence threshold, source diversity - Only then promote to durable preference

New Files: - /root/panchi_backend/app/shunya/context/human.py — HumanSignal, SignalStore, PatternCandidate - /root/panchi_backend/app/shunya/context/init.py

Database Migration: - CREATE TABLE human_signals (id, person_id, tenant_id, signal_type, value, payload, source_interaction, source_type, observed_at, valid_until, temporal_relevance, sensitivity, confidence) - CREATE TABLE pattern_candidates (id, person_id, signal_type, evidence_count, temporal_spread, confidence, status, promoted_at)

Boundary: HumanContext must check MemoryEligibility before persisting any signal.

PHASE 6 — Evidence, Provenance & Source Intelligence

Evidence (Architecture Report §7, §11): - provenance: MISSING - freshness: MISSING - source: 43 references but NOT structured as first-class concept - source_type: NOT formalized - KnowledgeFact has versioning but NO source tracking - web_search.py has Serper + DuckDuckGo but NO source normalization

Target (Master Directive §Phase 6): - SOURCE — where information came from - EVIDENCE — what was retrieved - FACT — what the organization knows - INFERENCE — what was derived

New Files: - /root/panchi_backend/app/shunya/evidence/models.py — Source, Evidence, Fact, Inference models - /root/panchi_backend/app/shunya/evidence/provenance.py — provenance chain tracker - /root/panchi_backend/app/shunya/evidence/init.py

Database Migration: - CREATE TABLE sources (id, source_type, url, authority, publication_time, retrieval_time, freshness_score) - CREATE TABLE evidences (id, source_id, fact_id, raw_content, confidence, retrieved_at) - Add evidence_id to knowledge_facts

PHASE 7 — Source/Evidence/Fact/Inference Runtime Distinction

Target (Master Directive §Phase 7): - SHUNYA must present: "AS PER Panchi Club DATA" "AS PER CURRENT EXTERNAL INFORMATION" "MY

ANALYSIS" "MY RECOMMENDATION" - These are semantically different outputs - Distinction exists in runtime data model, not just prompt wording

New Files: - /root/panchi_backend/app/shunya/evidence/runtime.py — EvidenceRuntime, OutputFormatter

Integration: - Extends Phase 6 with runtime query engine and presentation formatters - ReasoningContext gains evidence_chain property

PHASE 8 — LLM Intelligence Runtime

Evidence (Architecture Report §7): - VoiceProcessor is keyword routing — NOT LLM - CompanionEngine is rule-based — NOT LLM - CoachEngine is rule-based — NOT LLM - NO provider abstraction - NO tool-calling architecture - NO multi-step tool use

Target (Master Directive §Phase 8): - Provider-agnostic ModelProvider interface - ModelRequest, ModelResponse, ToolDefinition, ToolInvocation, ToolResult - ReasoningContext, IntelligenceRun - The LLM performs: intent understanding, context interpretation, tool selection, synthesis, reasoning assistance, language generation - SHUNYA architecture performs: identity, memory, retrieval, authorization, governance, persistence, provenance, execution, observation - Multi-step tool use support

New Files: - /root/panchi_backend/app/shunya/intelligence/provider.py — ModelProvider ABC -
/root/panchi_backend/app/shunya/intelligence/openai.py — OpenAI provider - /root/panchi_backend/app/shunya/intelligence/anthropic.py — Anthropic provider (optional) - /root/panchi_backend/app/shunya/intelligence/tools.py — ToolDefinition, ToolRegistry -
/root/panchi_backend/app/shunya/intelligence/request.py — ModelRequest, ModelResponse, ReasoningContext -
/root/panchi_backend/app/shunya/intelligence/init.py

Preserved: - VoiceProcessor keyword router remains for deterministic commands - LLM path selected by IntelligenceRouter based on intent complexity

PHASE 9 — Context Fusion Engine

Evidence (Architecture Report §8): - ShunyaInterface chains: KnowledgeLayer → ReasoningLayer → PlannerLayer - NO ContextLayer before ReasoningLayer - Architecture Report RECOMMENDED adding ContextLayer

Target (Master Directive §Phase 9): - ContextFusionEngine — NOT a simple context dictionary - Inputs: HUMAN_CONTEXT, ORGANIZATION_CONTEXT, RELATIONSHIP_CONTEXT, CONVERSATION_CONTEXT, BUSINESS_OBJECT_CONTEXT, WORLD_CONTEXT, TEMPORAL_CONTEXT, AUTHORITY_CONTEXT, PRIVACY_CONTEXT - Output: ReasoningContext (bounded, relevant, explainable) - Must answer: WHO is interacting? WHICH organization? WHAT role? WHO/WHAT are they talking about? WHAT conversation entities are active? WHAT does the company know? WHAT relationship history is relevant? WHAT human context is eligible? WHAT current world information is required? WHAT information is stale? WHAT conflicts exist? WHAT actions are authorized? WHAT requires approval?

New Files: - /root/panchi_backend/app/shunya/context/fusion.py — ContextFusionEngine - /root/panchi_backend/app/shunya/context/fusion/init.py

Integration: - Insert ContextFusionEngine between ShunyaInterface and ReasoningLayer - ShunyaInterface.process_message() → context_fusion() → reasoning()

PHASE 10 — Internal-First Knowledge Resolution

Evidence (Architecture Report §11): - web_search.py EXISTS (Serper + DuckDuckGo) - web_search_available() EXISTS - knowledge_store.py EXISTS (immutable fact store) - NO sufficiency evaluation - NO freshness evaluation - NO company-first then external pattern

Target (Master Directive §Phase 10): Resolution policy: - USER INTENT → CONTEXT FUSION → INTERNAL COMPANY KNOWLEDGE RETRIEVAL → SUFFICIENCY EVALUATION → FRESHNESS REQUIREMENT EVALUATION - IF SUFFICIENT AND CURRENT → REASON - IF INSUFFICIENT OR FRESHNESS-SENSITIVE → EXTERNAL RETRIEVAL → SOURCE NORMALIZATION → CONFLICT DETECTION → EVIDENCE FUSION → REASONING

New Files: - /root/panchi_backend/app/shunya/knowledge/sufficiency.py — KnowledgeSufficiencyEvaluator -
/root/panchi_backend/app/shunya/knowledge/freshness.py — FreshnessRequirementEvaluator

Integration: - Insert before web_search.py calls in pipeline - KnowledgeLayer gains sufficiency_evaluate() and freshness_evaluate()

PHASE 11 — World Intelligence Domain

Evidence (Architecture Report §11): - NO news APIs - NO RSS ingestion - NO scheduled intelligence jobs - NO source freshness evaluation - NO conflict detection - web_search.py EXISTS (Serper + DuckDuckGo) — EXTEND, not replace

Target (Master Directive §Phase 11): - On-demand retrieval - Freshness-sensitive retrieval - Future scheduled monitoring - Industry intelligence - External source adapters with normalization - Source identity, URL, publication time, retrieval time, freshness, authority, topic, geographic relevance

New Files: - /root/panchi_backend/app/shunya/world_intelligence/adapters.py — ExternalSourceAdapter, SerperAdapter, NewsAdapter, RSSAdapter - /root/panchi_backend/app/shunya/world_intelligence/normalizer.py — SourceNormalizer -
/root/panchi_backend/app/shunya/world_intelligence/init.py

PHASE 12 — Relevance Intelligence

Target (Master Directive §Phase 12): - SHUNYA must not become a news application - Relevance evaluation per external development: organization relevance, industry relevance, role relevance, current work relevance, person interest relevance, urgency, novelty, actionability - Conversational delivery: "By the way, there are two travel updates worth knowing today" — NOT "Here are 47 latest news articles"

New Files: - /root/panchi_backend/app/shunya/world_intelligence/relevance.py — RelevanceEvaluator

PHASE 13 — Persistent Plans & Governed Action

Evidence (Architecture Report §8): - PlannerLayer creates in-memory plans ONLY - Plans lost on process restart - GovernanceLayer EXISTS but minimal

Target (Master Directive §Phase 13): - Plan (id, tenant_id, requested_by, context_id, evidence_used, reasoning_output, status, created_at) - PlanStep (id, plan_id, sequence, action, state, assigned_to) - PlanEvidence (id, plan_id, evidence_id, role) - PlanDecision (id, plan_id, decision, decided_by, decided_at) - PlanApproval (id, plan_id, required_authority, granted_by, granted_at, state) - PlanExecution (id, plan_id, step_id, result, observed_at) - Principle: AI PROPOSES. AUTHORIZED GOVERNANCE DISPOSES. RUNTIME EXECUTES.

Files Affected: - /root/panchi_backend/app/models.py — add Plan, PlanStep, PlanEvidence, PlanDecision, PlanApproval, PlanExecution - /root/panchi_backend/app/shunya/planner.py — persist output - /root/panchi_backend/app/shunya/governance.py — strengthen with PlanApproval

Database Migration: - CREATE TABLE plans - CREATE TABLE plan_steps - CREATE TABLE plan_evidences - CREATE TABLE plan_decisions - CREATE TABLE plan_approvals - CREATE TABLE plan_executions

PHASE 14 — Close the Learning Loop

Evidence (Architecture Report §8): - OBSERVATION → LearningEntry → STOP - NO feedback to Knowledge or Reasoning - Learning is disconnected

Target (Master Directive §Phase 14): - EXECUTION → OBSERVATION → OUTCOME EVALUATION → LEARNING CANDIDATE → VALIDATION → KNOWLEDGE UPDATE OR PATTERN UPDATE → FUTURE REASONING - Learning candidate states: OBSERVED, REPEATED, SUPPORTED, CONFLICTED, VALIDATED, PROMOTED, REJECTED - PatternCandidate before promoting to company truth

Files Affected: - /root/panchi_backend/app/shunya/observer_learning.py — add candidate states, validation pipeline, feedback path - /root/panchi_backend/app/shunya/knowledge_store.py — receive validated learning from ObservationLayer

PHASE 15 — Relationship-Aware SHUNYA Assistant

Evidence (Architecture Report §7, §9): - CompanionEngine is rule-based greetings - CoachEngine is rule-based insights - NO relationship context - NO human context - NO conversation memory

Target (Master Directive §Phase 15): - Assistant adapts communication style based on eligible relationship context - Understands mood, workload, interests, concerns, preferences, ongoing work, customer relationship history - Employee example: "Aman looks strongest. Shall we clear those before lunch?" - Customer example: "Honestly Ritu, based on the kind of trips you have enjoyed earlier, I would not pack this Bali plan with sightseeing." - Must know WHY it made that statement - Human signals are not automatically business facts - Sensitive conversation is not automatically organizational memory

Files Affected: - /root/panchi_backend/app/shunya/companion.py — evolved CompanionEngine with relationship awareness, signal integration - /root/panchi_backend/app/shunya/coach.py — evolved CoachEngine with relationship context

PHASE 16 — Persistent SHUNYA Application Shell

Evidence (Architecture Report §3): - Page-oriented — NOT application-shell-oriented - Every navigation → full page load → layout remount → context loss → state reset - hx-boost=true on main element enables partial page updates - BUT shell (nav, sidebar) is re-rendered on every request - Three base templates: base.html, base_universal.html, auth_base.html

Target (Master Directive §Phase 16): - Persistent SHUNYA APP SHELL - Regions: Global Navigation, Active Organization Context, Assistant/Command Surface, Notification Surface, Active Workspace, Context/Intelligence Surface - Normal navigation updates ACTIVE WORKSPACE in place - NO full document reload, NO shell remount, NO context loss, NO AI conversation reset, NO unnecessary state reset - HTMX workspace-fragment responses from server routes - Alpine stores for: active workspace, navigation state, active organization, active conversation, active entity context, panel state, assistant state, mobile shell state - Browser history/back must remain sensible - Deep links must continue to work

Files Affected: - /root/shunya_os/templates/base.html — refactor into persistent shell - /root/shunya_os/templates/base_universal.html — refactor into persistent shell - /root/panchi_backend/app/routes.py — workspace-fragment response pattern - New: /root/shunya_os/static/js/shell.js — Alpine stores

Preserved: - Flask, Jinja2, HTMX, Alpine.js, PWA, theme engine — all capable of supporting persistent shell - Do NOT rewrite into React merely to call it an app

PHASE 17 — Mobile Application Runtime

Evidence (Architecture Report §4): - Sidebar on base_universal.html gets stuck open on mobile - 50KB response truncation issue - Content flow gaps - NO virtual keyboard handling - PWA manifest with "display": "standalone", "orientation": "portrait-primary"

Target (Master Directive §Phase 17): - At application initialization resolve: DEVICE CLASS, VIEWPORT CLASS, DISPLAY MODE, ORIENTATION CAPABILITY, SAFE AREA, INPUT MODE - Resolve mobile application shell early — do not repeatedly restructure as user navigates - Portrait-first, stable shell, bottom nav persistent, active workspace scroll ownership - Assistant accessible globally - No stuck sidebar, no desktop shell bleed, no fixed overlay collisions, no context loss during workspace changes - Touch-friendly targets - Virtual keyboard handling - Orientation locking only where runtime permits — portrait-first layout policy in all modes

Files Affected: - /root/shunya_os/templates/base.html — mobile shell refinement - /root/shunya_os/static/shunya.css — mobile runtime CSS - /root/shunya_os/static/manifest.json — preserve standalone + portrait-primary - /root/shunya_os/static/sw.js — update cache strategy if needed

Risk Register

Phase	Risk	Likelihood	Impact	Mitigation
0	Uncommitted work lost	Low	High	Git commit + tag before changes
1	Identity resolution breaks lead flow	Medium	Medium	Person FK is nullable; adapter preserves string access
1	Ambiguous identity merges wrong persons	Medium	High	Never silently merge; AMBIGUOUS resolution blocks
3	Conversation context storage overhead	Low	Low	TTL-based cleanup; session vs long-term distinction
5	Privacy policy gaps for Indian users	Medium	Medium	Baseline GDPR-style; extend for Indian IT Act
8	LLM API cost at scale	Medium	Medium	Provider-agnostic; local models as fallback
11-12	External data quality unreliable	Medium	Low	Source authority ranking, conflict detection
14	Automated learning promotes false patterns	Medium	High	Learning candidate validation chain; human review gate
16	HTMX shell transition breaks existing pages	Medium	High	Workspace-fragment responses; test every route
16	Persistent state conflicts with server truth	Medium	Medium	Alpine stores for UI state only; server remains source of truth

Rollback Strategy

Every phase creates a git tag:

pre-evolution-baseline phase-1-complete phase-2-complete ...

Rollback via git checkout pre-evolution-baseline restores 100% production functionality.

The original panchi.service systemd unit is NEVER modified by evolution work — only its Python source changes. Nginx, PostgreSQL, systemd configuration remain stable.

All database migrations are reversible (ADD COLUMN is nullable; DROP TABLE has CREATE TABLE in rollback).

Implementation Discipline

Per Master Directive \$Implementation Discipline:

For each phase: 1. Inspect existing implementation 2. Preserve working behaviour 3. Design minimum first-class domain 4. Add database migration 5. Add compatibility adapter 6. Add tests 7. Integrate into runtime 8. Validate existing flows 9. Update architecture documentation

A domain/package exists only when: - It owns a clear responsibility - It has executable implementation - It is integrated into runtime - It has tests

No empty scaffolding. No cosmetic-first implementation. The objective is not more code. The objective is a coherent SHUNYA.